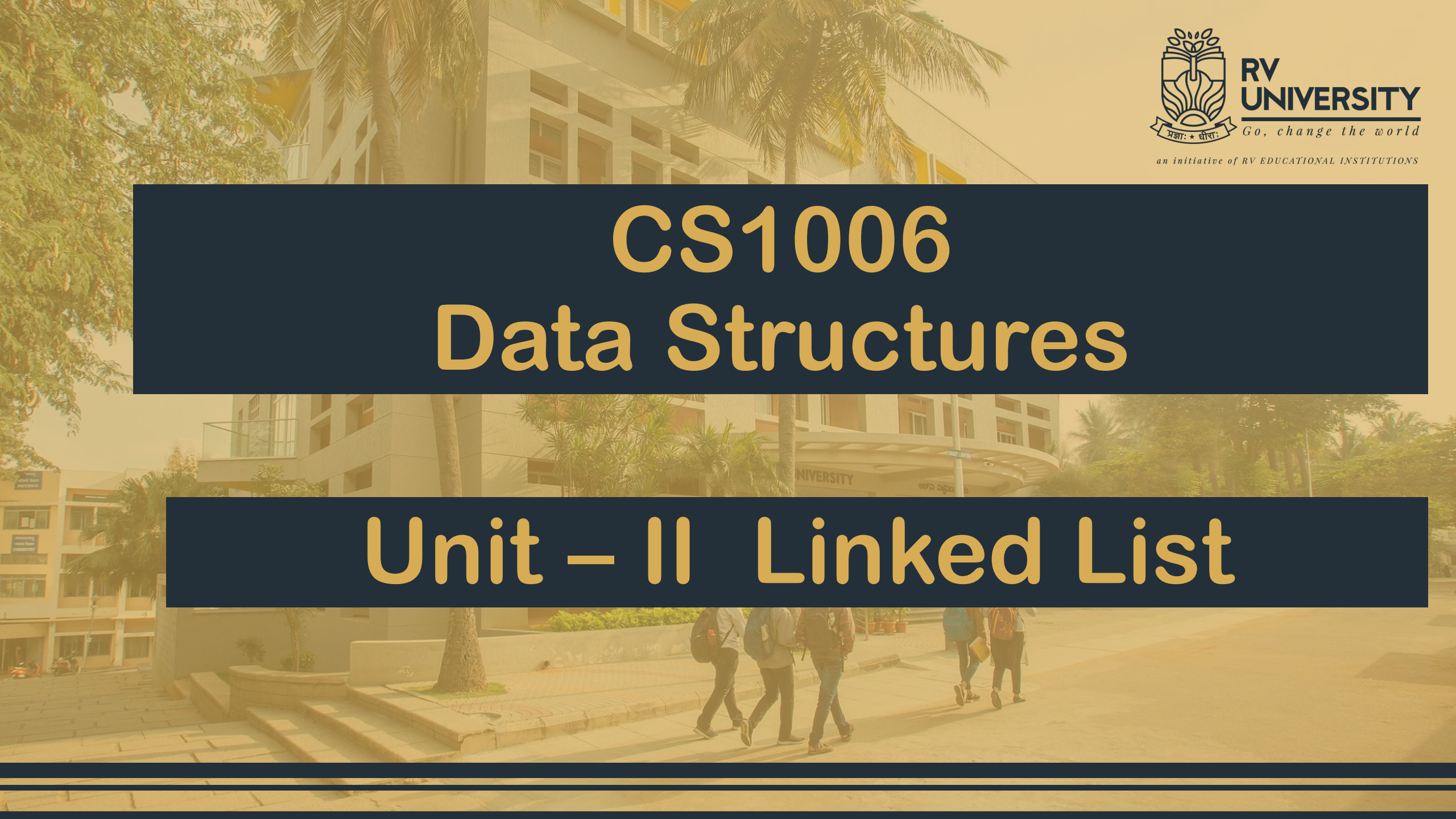


RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

CS1006
Data Structures

Unit – II Linked List

Syllabus

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Unit II: Linked List

9 hours

- **Linked Lists**
  - ✔ Singly,
  - ✔ Doubly,
  - ✔ Circular Singly,
  - ✔ Circular Doubly
- **Implementation of Operations –**
  - ✔ Create,
  - ✔ Insert,
  - ✔ Traverse,
  - ✔ Delete,
  - ✔ Search and
  - ✔ Update

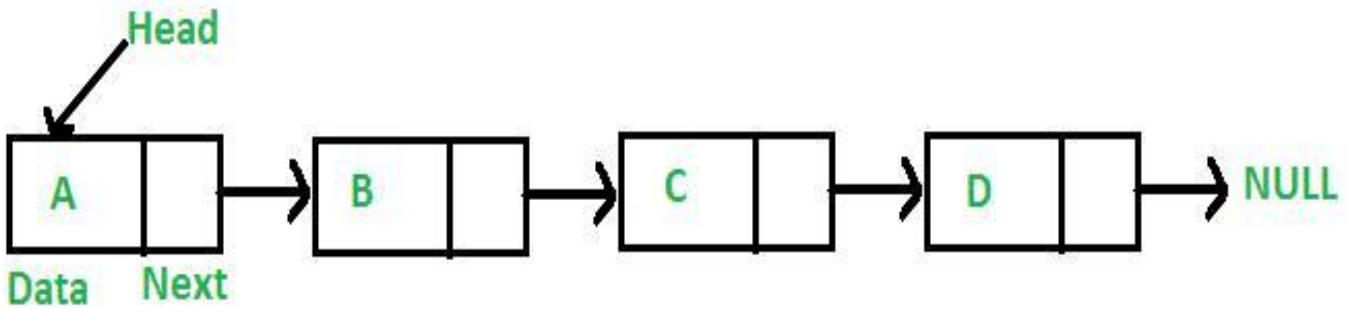
Implementation of List ADT

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- A linked list is a *sequential access* data structure, where each element can be accessed only in particular order.
  - A typical illustration of sequential access is a roll of paper or tape - all prior material must be unrolled in order to get to data you want.

Linked List Implementation – List

- Like arrays, Linked List is a linear data structure.
- Unlike arrays, linked list elements are not stored at contiguous location; the elements are linked using pointers.



Need of Linked List

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Arrays can be used to store linear data of similar types, but arrays have following limitations.

- The size of the arrays is fixed: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage.

- Inserting a new element in an array of elements is expensive, because room has to be created for the new elements and to create room existing elements have to shifted.

Implications of Array

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Advantages over arrays

- ✔ Dynamic size
- ✔ Ease of insertion/deletion

Drawbacks

- ✔ Random access is not allowed. We have to access elements sequentially starting from the first node. So we cannot do binary search with linked lists.
- ✔ Extra memory space for a pointer is required with each element of the list.

Representation of Linked List in 'C'

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- A linked list is represented by a pointer to the first node of the linked list. The first node is called head. If the linked list is empty, then value of head is NULL.
- Each node in a list consists of at least two parts:

✔ data

✔ pointer to the next node

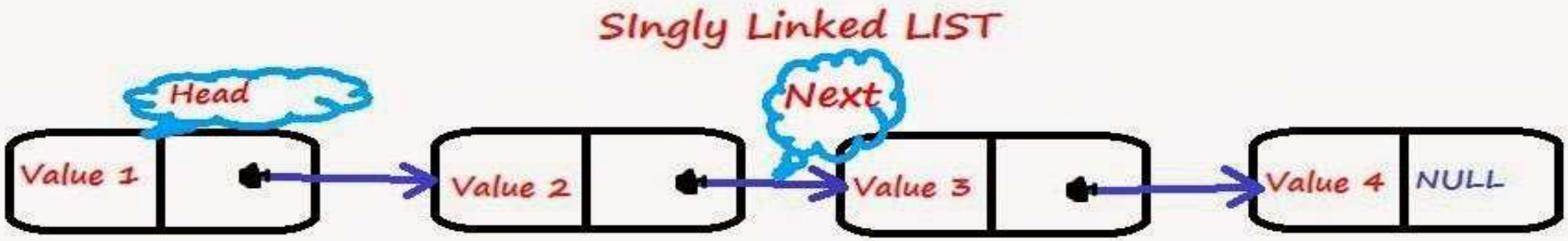
- In C, we can represent a node using structures.

Representation of Linked List in 'C'

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

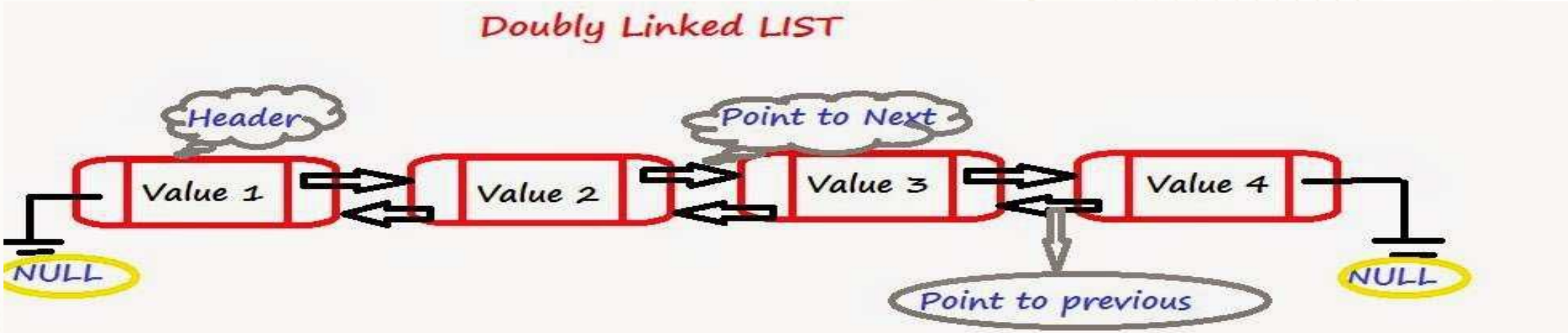
```
        Type declaration for linked list
#ifndef _List_H
struct Node;  //Variable declarations
typedef struct Node *PtrToNode;
typedef PtrToNode List;
typedef PtrToNode Position;

List MakeEmpty(List L); //Function Declarations
int IsEmpty(List L);
int IsLast(Position P, List L);
Position Find(ElementType X, List L);
void Delete (ElementType X, List L);
Position FindPrevious(ElementType X, List L);
void Insert(ElementType X, List L, Position P);
void DeleteList(List L);
Position Header(List L);
Position First(List L);
Position Advance(Position P);
ElementType Retrieve(Position P);

#endif /* _List_H */
  //Strcture of Node declaration
 struct Node
 {
        ElementType Element;
        Position Next;
 }
```

Singly Linked List



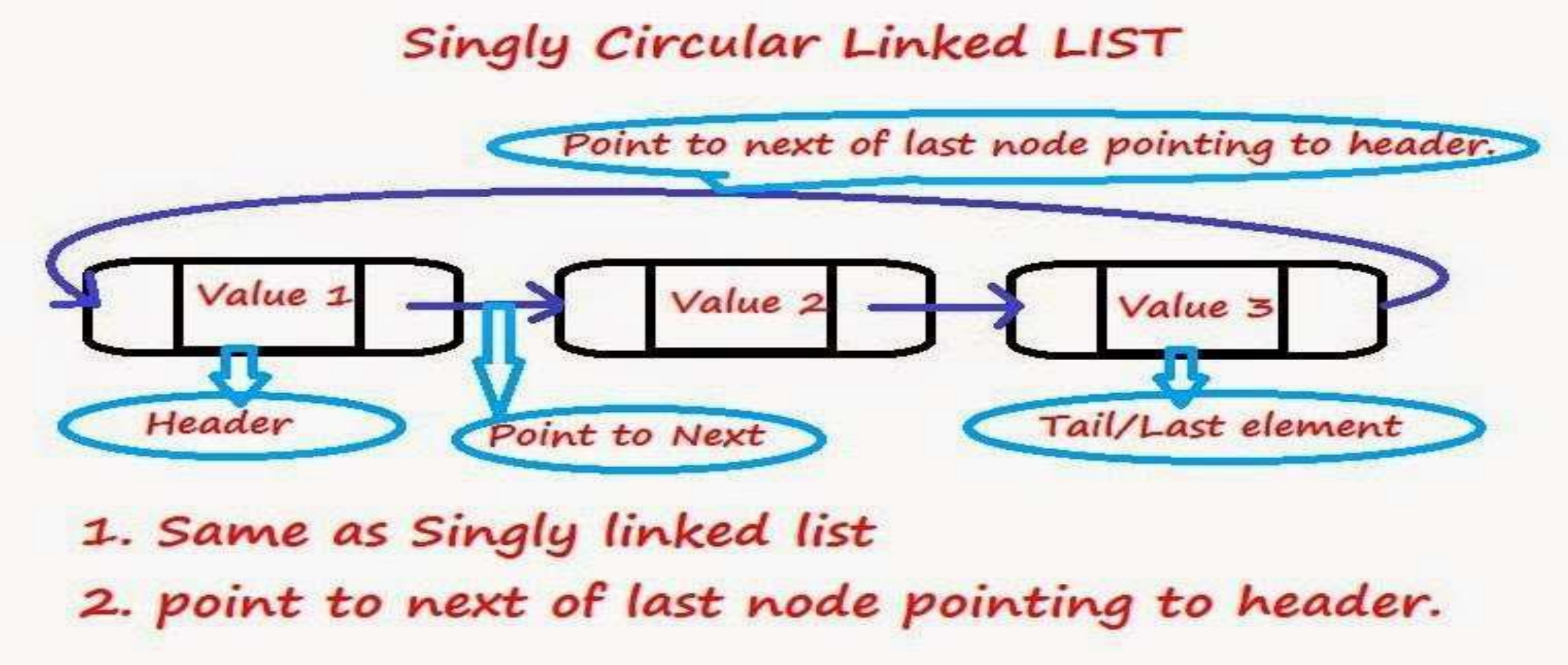
1. Each Element/Node of a list comprising of 2 items (Data, Reference to next node)
2. The last node has a reference to null.
3. The entry point into a linked list is called the head of the list. It should be noted that head is not a separate node, but the reference to the first node.
4. If the list is empty then the head is a null reference.

Doubly Linked List

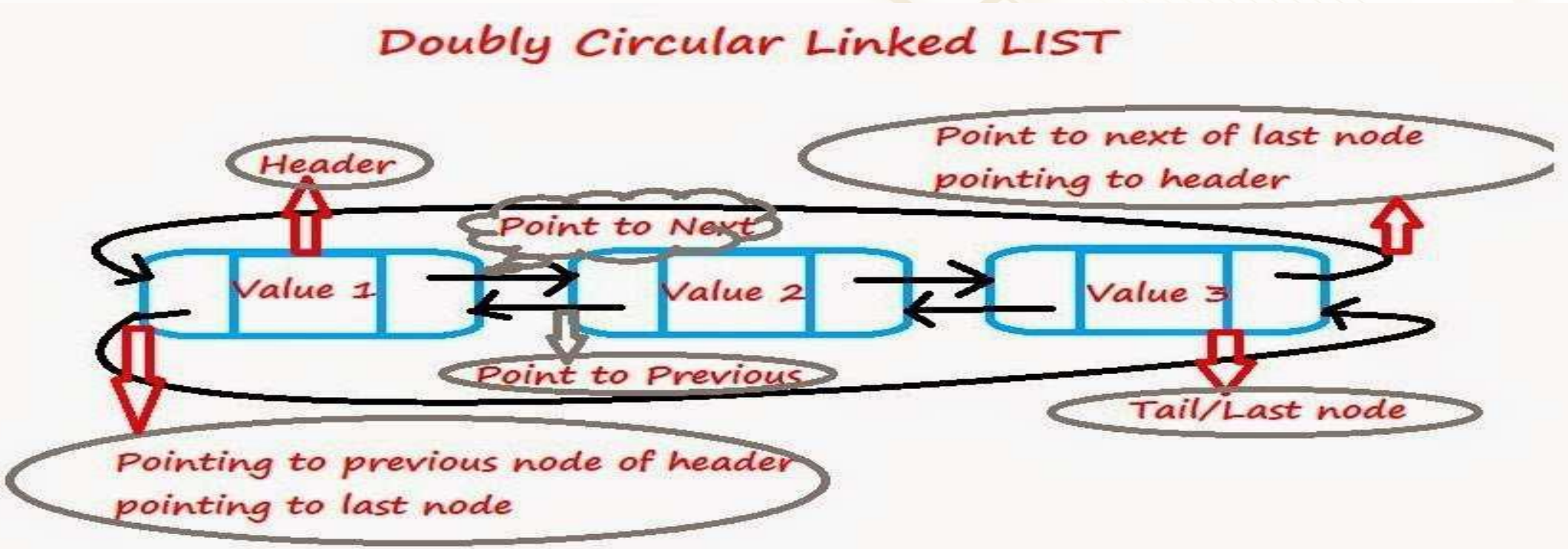


1. Each node/element of a list comprising of 3 items (Data, Reference to next node & Reference to previous node)
2. The Reference to next node of a last node is NULL.
3. The Reference to previous node of a header node is NULL.
4. Two way access is possible. We can start accessing nodes from start as well as from last.

Singly Circular Linked List



Doubly Circular Linked List



1. Same as doubly linked list.
2. point to next of last node pointing to header.
3. point to previous of header pointing to last node.

Comparison

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

S.No	Array Based	Linked List
1	No Wasted Space for an individual element	Only need space for the objects actually on the list
2	Size must be predetermined; wasted space for lists with empty slots	Overhead for links(an extra pointer added to every node)
3	Ω(n) (or greater)	Θ(n)

Applications of List

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Lists can be used to store a list of elements. However, unlike in traditional arrays, lists can expand and shrink, and are stored dynamically in memory.

- In computing, lists are easier to implement than sets.

- Lists also form the basis for other abstract data types including the queue, the stack, and their variations.

Polynomial Manipulation

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- A polynomial is a mathematical expression consisting of a sum of terms, each term including a variable or variables raised to a power and multiplied by a coefficient. The simplest polynomials have one variable.
- **Representation of a Polynomial:** A polynomial is an expression that contains more than two terms. A term is made up of coefficient and exponent. An example of polynomial is

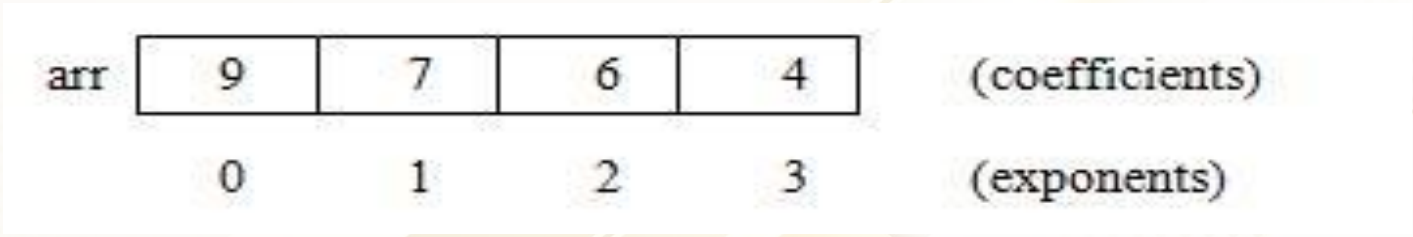
- $P(x) = 4x^3+6x^2+7x+9$

$$f(x) = \sum_{i=0}^{n} a_i x^i$$

Polynomial Manipulation

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- A polynomial thus may be represented using arrays or linked lists.
- Array representation assumes that the exponents of the given expression are arranged from 0 to the highest value (degree), which is represented by the subscript of the array beginning with 0.
- The coefficients of the respective exponent are placed at an appropriate index in the array.

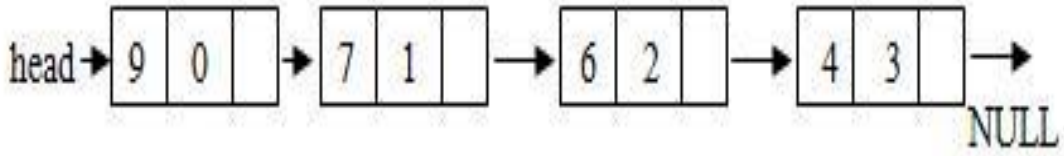


Polynomial Structure

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

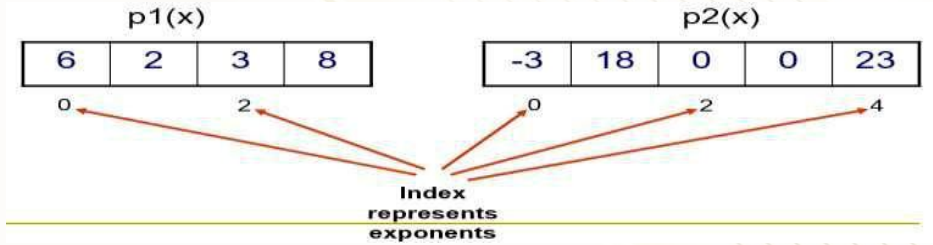
- A polynomial may also be represented using a linked list. A structure may be defined such that it contains two parts- one is the coefficient and second is the corresponding exponent. The structure definition may be given as shown below:

```
struct polynomial
{
  int coefficient;
  int exponent;
  struct polynomial *next;
};
```



Polynomial Operations

The most common operations on a polynomial:
Add & Subtract
Multiply
Differentiate
Integrate
etc…



There are different ways of implementing the polynomial ADT:
Array (not recommended)
Linked List (preferred and recommended)
Array Implementation:

$p1(x) = 8x^3 + 3x^2 + 2x + 6$
$p2(x) = 23x^4 + 18x - 3$

This is why arrays aren't good to represent polynomials:
$p3(x) = 16x^{21} - 3x^5 + 2x + 6$

Polynomial using Arrays

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Advantages of using an Array:

- only good for non-sparse polynomials.
- ease of storage and retrieval.

Disadvantages of using an Array:

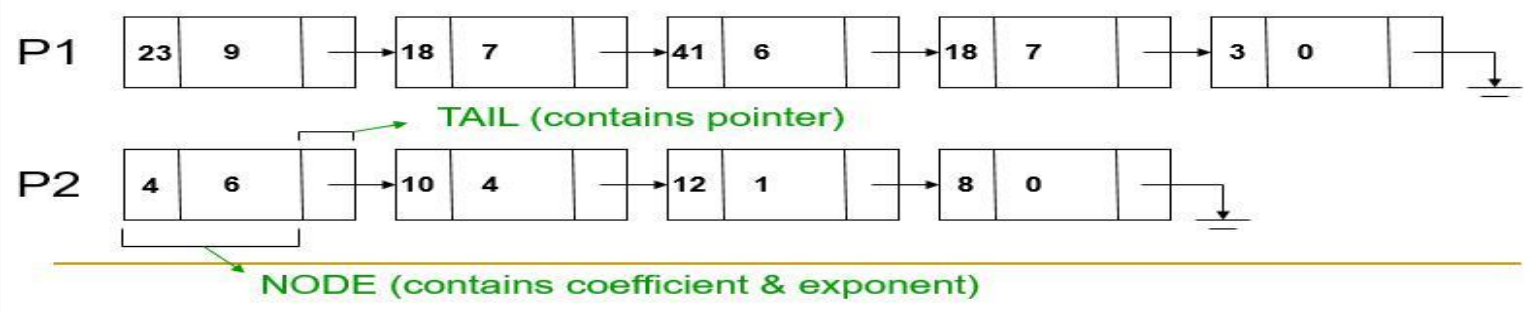
- Have to allocate array size ahead of time. Huge array size required for sparse polynomials.
- Waste of space and runtime.

Polynomial using Linked List

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

$p1(x) = 23x^9 + 18x^7 + 41x^6 + 163x^4 + 3$

$p2(x) = 4x^6 + 10x^4 + 12x + 8$




```
struct polynode {
        int coef;
        int exp;
        struct polynode * next;
  };
typedef struct polynode *polyptr;
```

Polynomial using Linked List

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Advantages of using a Linked list:

- save space (don't have to worry about sparse polynomials) and easy to maintain
- don't need to allocate list size and can declare nodes (terms) only as needed

Disadvantages of using a Linked list :

- can't go backwards through the list
- can't jump to the beginning of the list from the end.

Polynomial Addition

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

For adding two polynomials using arrays is straightforward method, since both the arrays may be added up element wise beginning from 0 to n-1, resulting in addition of two polynomials.

Addition of two polynomials using linked list requires comparing the exponents, and wherever the exponents are found to be same, the coefficients are added up.

For terms with different exponents, the complete term is simply added to the result thereby making it a part of addition result.

$$\begin{array}{r} x^5 + 3x^3 + 4 \\ +\ 6x^6 + \quad + 4x^3 \\ \hline 6x^6 + x^5 + 7x^3 + 4 \end{array}$$

Polynomial Addition

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Adding polynomials using a Linked list representation: (storing the result in p3)
- To do this, we have to break the process down to cases:

Case 1: exponent of p1 > exponent of p2

- Copy node of p1 to end of p3.

- [go to next node]

Case 2: exponent of p1 < exponent of p2

- Copy node of p2 to end of p3.

- [go to next node]

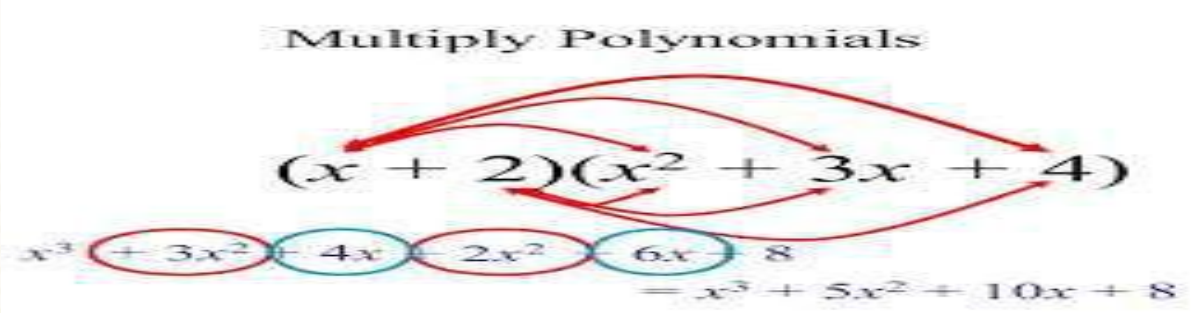
Case 3: exponent of p1 = exponent of p2

- Create a new node in p3 with the same exponent and with the sum of the coefficients of p1 and p2.

Polynomial Multiplication

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Multiplication of two polynomials however requires manipulation of each node such that the exponents are added up and the coefficients are multiplied.
- After each term of first polynomial is operated upon with each term of the second polynomial, then the result has to be added up by comparing the exponents and adding the coefficients for similar exponents and including terms as such with dissimilar exponents in the result.



Polynomial operations

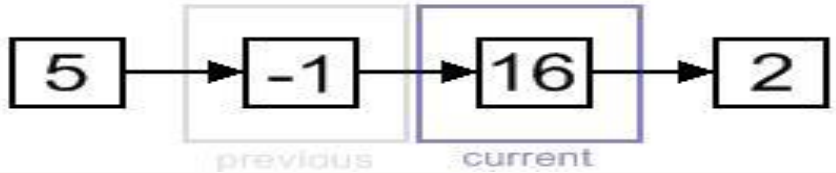
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- ✔ **Insertion**
- ✔ **Deletion**
- ✔ **Merge two sorted linked lists**
- ✔ **Traversal**
  - Assume, that we have a list with some nodes. Traversal is the very basic operation, which presents as a part in almost every operation on a singly-linked list.
  - For instance, algorithm may traverse a singly-linked list to find a value, find a position for insertion, etc. For a singly-linked list, only forward direction traversal is possible.

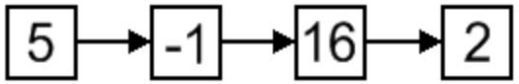
Traversal algorithm

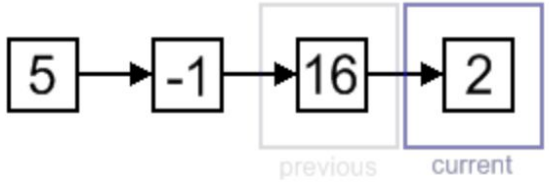
- Beginning from the head, check, if the end of a list hasn't been reached yet;
- do some actions with the current node, which is specific for particular algorithm; current node becomes previous and –next node becomes current. Go to the step 1.

Polynomial Operations

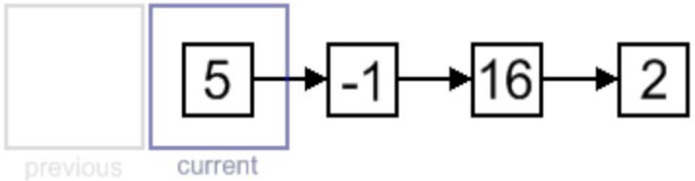


step 2
sum = 4

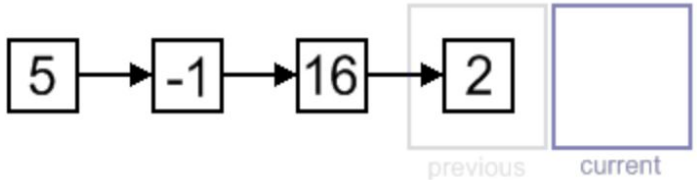




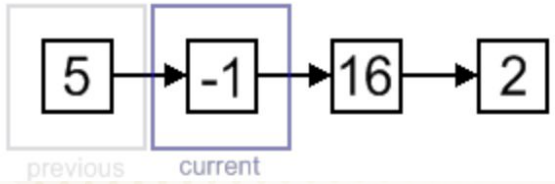
step 3
sum = 20



initial state
sum = 0



final state
sum = 22



step 1
sum = 5

External References

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

1. https://nptel.ac.in/courses/106102064/
2. https://nptel.ac.in/courses/106103069/
3. https://nptel.ac.in/courses/106106127/
4. https://nptel.ac.in/courses/106106130/
5. https://www.geeksforgeeks.org/data-structures/
6. https://www.javatpoint.com/data-structure-tutorial
7. https://www.coursera.org/learn/data-structures
8. https://www.studytonight.com/data-structures/introduction-to-data-structures
9. https://www.youtube.com/watch?v=egb6qFw42JM (Tamil videos)


RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS
Thank You